

University of Pisa
Master's Degree in Computer Engineering
Distributed Systems and Middleware Technologies

VoltShare

Project Documentation

A distributed coordination system for electric-vehicle charging stations

Group members:

Nome Cognome
Nome Cognome

Academic Year 2025/2026

Contents

1	Introduction	2
2	Requirements Specification	2
2.1	Functional requirements	2
2.2	Non-functional requirements	2
3	Domain Rules	2
4	System Architecture	2
4.1	Key design decisions	2
4.2	Architectural components	2
5	Synchronization and Coordination Problems	3
6	Architecture Protocols	3
7	Addressing the Problems	3
7.1	Mutual exclusion on a connector (P1)	3
7.2	One reservation per vehicle, network-wide (P2)	4
7.2.1	Claim first, commit after	4
7.2.2	Why a central arbiter	4
7.2.3	Ordering decided by one clock	5
7.3	Leases, and commands that arrive twice (P3, P7)	5
7.4	Coordinator replication: election and quorum (P2b)	6
7.5	Sharing the site power (P5)	6
8	Fault Tolerance and Recovery	7
8.1	The failure model	7
8.2	Two failure detectors, because there are two failures	7
8.3	A station dies, and a station goes silent	8
8.4	Rebuilding the claim table after a failover	9
8.5	A charge point dies: grace, then honesty	10
8.6	Supervision inside a node	11
8.7	Three delivery guarantees on one wire	11
8.8	What tolerates nothing, and is declared	12
9	API and Protocol Specification	12
10	Deployment	12
11	Testing and Demonstration	12
12	Future Works	13

1 Introduction

[to be written — 1.5 pages. What the system does, for whom, and which distributed problems it solves. Must answer within the first page why this domain has genuine contention: the connector is a physically exclusive resource, held for a long and imprecisely known time, and drivers are autonomous agents the operator can only arbitrate. Sources: SCOPE §1, DESIGN-NOTES §1. Close on a short list of the problems the project sets out to solve, as BlackNet does.]

2 Requirements Specification

2.1 Functional requirements

[to be written — 1.5 pages. Account and vehicle, station discovery, reservation, charging session, power allocation, billing and history, notifications. Source: SCOPE §3. Bullet lists are legitimate here: the material is genuinely enumerable.]

2.2 Non-functional requirements

[to be written — 1 page. Exclusive use, no resource lost to a failure, station autonomy, real time by server push, horizontal growth. Source: SCOPE §4.]

3 Domain Rules

[to be written — 1.5 pages. The rules the system enforces, needed to follow the sections that come after: reservation under a lease, no-show and suspension, session authorisation against the reservation, overstay and grace period, power sharing within the site budget, tariffs. Sources: SCOPE §3.3–3.6. Keep it short: these are rules, not discussion. The equivalent of BlackNet's "Game Rules".]

4 System Architecture

4.1 Key design decisions

[to be written — 3 pages. One subsection per decision, each with choice, mechanism, discarded alternative and reason. The decisions: server-side rendering instead of an SPA; why middleware rather than plain TCP sockets (distributed Erlang, JInterface, WebSocket), which the course requires to be argued; coordinators separate from stations, and the site controller living at the station; volatile state in Erlang memory against durable state in MySQL; the charge-point emulator and the OCPP 1.6-J subset that fixes the system boundary. Sources: SCOPE §6, §7, §9, DESIGN-NOTES §1, §5.]

4.2 Architectural components

[to be written — 2 pages. The seven nodes: back office (Java/Tomcat), three coordinators (Erlang/OTP), two station controllers (Erlang/OTP + Cowboy), MySQL. Technology, responsibility and runtime behaviour for each. The deployment figure goes here. Sources: SCOPE §6, deploy/docker-compose.yml.]

5 Synchronization and Coordination Problems

[to be written — 2.5 pages. The problems only, no solutions: those are section 7 and section 8. One paragraph per problem P1–P7, saying where it arises, why it is hard, and what would happen without a solution. The division that must come across is between problems inside one station (P1, P5, P6) and problems across nodes (P2, P2b, P4). Close on the communication problems: heterogeneous runtimes exchanging structured data, long-lived low-latency connections to many clients, asynchronous state propagation without polling. Source: SCOPE §5, DESIGN-NOTES §2.]

6 Architecture Protocols

[to be written — 1.5 pages. Which protocol runs on which edge and why that one, split into client-facing and internal as BlackNet does. HTTP browser to back office; WebSocket browser to station; WebSocket/JSON charge point to station; distributed Erlang station to coordinator; JInterface back office to coordinator; SQL to MySQL. Overview only: the message sets are specified in section 9. Sources: SCOPE §6, contracts/.]

7 Addressing the Problems

Section 5 listed the problems. This section describes the mechanism chosen for each one, the alternatives that were set aside, and what each choice costs. Failures have a section of their own (section 8), because there are five different mechanisms answering them, one per failure.

Two of the problems are settled inside a single station and two of them are not, and the difference decides everything else. A connector belongs to exactly one station, always. No other node can grant it, so no remote agreement is needed and the station can decide alone. A vehicle is the only entity that can appear at two stations at the same time: the driver opens two browser tabs and presses *reserve* on both. Each station checks its own connector, finds it free, and grants. Both are locally correct, both respected the exclusivity of their own outlet, and the network-wide invariant is broken. Neither station can detect this by itself, because the missing fact is held by neither of them.

7.1 Mutual exclusion on a connector (P1)

Each physical connector is owned by one Erlang process, a `gen_statem` implemented in `vs_connector`. Every request that touches that outlet becomes a message in that process's mailbox, and messages are handled one at a time. Two drivers reserving the same connector in the same millisecond are therefore serialised by the runtime, and the second one finds the connector already in state `held`.

There is no lock in the station code, and none is needed. This is the actor model applied to its textbook case: a resource paired with the process that guards it, so that mutual exclusion becomes a property of the mailbox instead of a discipline the programmer has to keep.

The state machine carries a second guarantee that is easy to overlook. A session cannot start on a connector held by somebody else, and no check anywhere in the code is what prevents it: `charging` is simply unreachable from `held` with the wrong vehicle. There is no branch to forget. Events that do not apply to the current state fail to match, and the connector answers with a refusal that costs one clause. Enforcement of this kind fails loudly during development, where a missing clause is a crash and therefore cheap.

7.2 One reservation per vehicle, network-wide (P2)

This is the guarantee the architecture is built around, and the mechanism is a *claim*: a permission, granted by the coordinator, recording that a given vehicle is now committed and to which station.

The two words matter throughout this document. A **reservation** is what the driver sees: connector 3 at this station, held for me until 18:07, owned by the connector process. A **claim** is what made it possible: an entry at the coordinator, about a vehicle, valid across the whole network. The reservation is local and concerns an outlet; the claim is global and concerns a car.

7.2.1 Claim first, commit after

A station obtains the claim *before* it creates the reservation, never the other way round. The order is the whole mechanism, and it is worth stating why.

In the wrong order, a crash between the two steps leaves a reservation that no other node knows about. The vehicle can then obtain a second reservation elsewhere, and the invariant is broken silently, with nothing in the system aware of it. In the right order, the same crash leaves a claim that was granted and never used. Nobody can reserve for that vehicle for a few minutes, and then the claim expires on its own. The system errs towards refusing rather than towards granting twice.

This is the reasoning behind a write-ahead log, applied to a permission instead of to a balance: record the commitment before acting on it, so that the recovery path only ever has to undo an intention.

7.2.2 Why a central arbiter

The coordinator is a single process, `vs_coord_srv`, holding the claim table in its own memory. Concurrent requests from different stations queue in one mailbox and are decided one at a time, which is what makes the decision serialisable at all. In the vocabulary of the course this is centralised distributed mutual exclusion, with the difference that the critical section is not one: there is one per vehicle, and they are independent.

That last point decided the algorithm. We considered four designs.

One coordinator, unreplicated. The simplest, and its failure is only partial: the coordinator sits outside the path that delivers power, so sessions in progress survive it. We rejected it because it leaves an avoidable single point of failure on the one guarantee the system exists to provide.

A UNIQUE constraint in MySQL. A uniqueness constraint on the vehicle would enforce P2 correctly and cost almost nothing to write. Two reasons against it. The coordinator has to exist anyway, for the cluster map and for node monitoring, so this option removes no component. And a claim that *expires by itself* is a timer inside a process, whereas in the database it becomes either a cleanup job or a filter on every read, which is polling: the one thing this architecture avoids on every other edge. It also moves the single point of failure onto the database instead of removing it.

Direct coordination between stations (Ricart-Agrawala). No central authority by construction, which is attractive. It costs $2(n - 1)$ messages per reservation, needs careful handling of nodes that die holding a token, and, decisively, it serialises access to *one* critical section. Here every vehicle is an independent one, and two reservations for two different vehicles have no reason to wait for each other.

Partitioning the vehicle space across coordinators. Technically the cleanest of the four: the node responsible for *hash(vehicle)* decides, no node sees all the traffic, and a failure blocks one shard instead of the network. We set it aside because every node must agree on the same partition map, the map must be rebalanced when membership changes, and each shard still needs its own failover. It adds to the chosen design rather than replacing it, and it is recorded as future work in section 12.

7.2.3 Ordering decided by one clock

When a claim is granted, the coordinator stamps it with `granted_at` and sends that timestamp back with the grant. The station stores it and echoes it in every renewal.

The purpose is conflict resolution after a failover. If two claims for the same vehicle ever meet in the table, the older one wins, and the comparison is between two timestamps produced by the *same* process. No two machines with unsynchronised clocks are ever compared. We did not need logical clocks to get this: the coordinator already serialises the decisions that matter, so a single physical clock, used at one place only, is sufficient and simpler to defend.

7.3 Leases, and commands that arrive twice (P3, P7)

A driver who reserves and never arrives is not a failure of the system, and it cannot be solved by asking the client to be well behaved. The answer is a lease: every reservation is granted with an expiry, and when the expiry passes the connector returns to `free` with no operator action and no cooperation from the party that disappeared.

There are two nested expiries, and they protect against two different absent parties:

- the **reservation lease** protects against the driver who never shows up. It is the timer the connector process holds;
- the **claim expiry** protects against the station that dies without releasing what it holds. Without it, a crashed node would lock its vehicles out of the entire network until somebody intervened.

The claim outlives the reservation on purpose, by `CLAIM_GRACE_SECONDS`, so that a station renewing slightly late does not lose a claim for a car that is charging. Stations refresh what they hold every `CLAIM_RENEW_INTERVAL_MS`, ten seconds by default. The default lease is fifteen minutes; the demonstration profile in `.env.demo` shortens it to 90 seconds so that a no-show can be watched from beginning to end without waiting a quarter of an hour. No clock is scaled: only the parameters change.

This is the leasing pattern of the course material, the same mechanism used by distributed garbage collectors to decide that a remote reference has gone away. The property that makes it work here is that the resource is released by the passage of time and not by a message, so no participant can hold it hostage by falling silent.

At-most-once for driver commands. A driver on a mobile connection sends `reserve`, sees nothing happen, and presses the button again. If both messages arrive, the naive outcome is two reservations, or a session started twice.

Every driver command therefore carries a mandatory `request_id`, and `vs_driver_proto` keeps a small cache of recent identifiers with the frames that answered them. A repeated identifier is answered with the stored reply and is never dispatched a second time. The retry is invisible to the connector, which is the point: the deduplication belongs at the edge, where the duplicate is created. The state machine never has to reason about it.

7.4 Coordinator replication: election and quorum (P2b)

Making the coordinator authoritative creates the problem of the coordinator itself failing. Three coordinator nodes run; one is the leader and is the only node that grants or refuses claims. A station that addresses one of the other two is redirected to the leader.

Election. When the leader stops responding, the survivors elect a new one with the bully algorithm over their fixed node identifiers: the highest live identifier wins. A node that notices the leader is gone sends **ELECTION** to the nodes above it. Silence means it is the most senior node alive and it announces **LEADER** downwards; an answer means somebody more senior is handling it, and it waits. With three nodes this settles in two rounds of messages. Detection is by timeout, which means we assume a *synchronous* system, an upper bound on response time. That assumption is stated here because the correctness of the detection depends on it, and it is the kind of thing an examiner is right to ask about.

Quorum. Bully alone does not prevent split brain. In a partition each side happily elects its own local maximum, and two authorities granting claims break precisely the invariant the coordinator exists to protect. So a coordinator serves only while it can see a majority of the configured coordinator cluster, itself included: two out of three. A node in the minority suspends itself and refuses everything, leader or not.

This is why the cluster has three members and not two. With an even number, a partition can produce two halves that each consider themselves entitled to grant.

The cost is real, and we state it: during a partition, the minority side refuses *new reservations*. Charging sessions already running are untouched, because the coordinator is not on the path that delivers power. It only decides who may hold a vehicle. Refusing to serve when the invariant cannot be guaranteed is deliberate, and it is the same instinct that runs through the lease and the claim expiry: when in doubt, withhold.

State after a failover. A new leader inherits nothing, and it does not inherit a replicated log either. It asks the stations what they hold and rebuilds the claim table from their answers. The reason this is possible, and the reason it is a design decision rather than a shortcut, is developed in section 8.4: the coordinator was built as an index over state that lives somewhere else.

7.5 Sharing the site power (P5)

The last problem is a different species from the others, and putting it next to them is deliberate. A connector is exclusive, so it is granted to one holder. The electrical capacity of a site is divisible, so it is not granted at all: it is apportioned, and the apportionment changes every time a car arrives, leaves, or changes what it can absorb. Mutual exclusion and permits are two separate chapters of the course, and this system needs both, inside the same node.

The site budget is lower than the sum of the nominal ratings of the connectors, which is true of real installations and is what makes the problem exist. **vs_power** recomputes the split on every arrival, every departure, and on a periodic tick that follows the charging curve as batteries fill.

The policy is max-min fair share with hand-back. Every active session starts from $budget/N$. A session that cannot absorb its share, either because the car is small or because it is tapering near full, takes only what it asks for and hands the surplus back, which is then split again among the others. The loop repeats until nobody hands anything back or the budget is exhausted.

We chose it over the two alternatives named in the requirements for what it yields and for how it can be defended. Proportional-to-demand does not deliver more energy: it saturates the same budget and only changes who receives what, so it has to be argued as a political choice about which cars matter more. Reservation-holders-first would starve a walk-in session down to suspension, which contradicts the rule that a no-show costs a driver the right to book while leaving them free to charge. Fair share with hand-back is one sentence long, and no kilowatt sits unused while somebody could absorb it.

The floor. Below a minimum viable power a session is suspended rather than starved, which the requirements ask for and which `vs_power` implements by allocating exactly 0.0 kW: the session stays open and draws nothing. The condition has two halves. A session is suspended when its share is under `MIN_CHARGE_KW` *and* under what it asked for. The floor exists to answer scarcity, so a car that receives exactly what it requested is not being starved by anybody, and taking it to zero would free nothing for anybody else.

When the budget really is short, the session suspended is the most recent arrival, ordered by `started_at` with the connector identifier breaking a tie. That rule is stable by construction, because `started_at` never changes. An earlier version ranked sessions by how far they were from their demand instead, and produced a two-tick oscillation: suspending a session zeroed its limit, a zero limit made it look demand-bound, and the next tick reversed the decision. The lesson generalises beyond this project. A control rule must not take as input a quantity that the rule itself sets.

8 Fault Tolerance and Recovery

Fault tolerance in this system is not one mechanism. It is five, chosen per failure, and the claim worth defending is that each was picked for a specific reason. None of them was applied across the board.

One idea organises them. Every component keeps only the state it owns, and every other component treats that state as recoverable from its owner. Tolerance follows from ownership rather than from replication, which is why there is no consensus protocol anywhere in this project. Its absence was decided, and section 8.4 gives the argument.

8.1 The failure model

We assume crash-stop and network partition. Byzantine behaviour is out of scope: a node that answers, answers truthfully. Table 1 lists what can fail and what the system does about it.

8.2 Two failure detectors, because there are two failures

This is the design decision in this section most worth defending.

Erlang gives us `net_kernel:monitor_nodes/1` for free, and it fires the instant a TCP connection breaks. That is exactly what a killed container looks like. It is immediate and it costs nothing.

But a node that stops answering *without* closing its socket is invisible to it. A network partition, a frozen virtual machine, a host that has started swapping: the connection is still open, so no `nodedown` is delivered until the distribution's own tick expires, and `net_ticktime` defaults to sixty seconds. Sixty seconds of a minority leader still granting claims is precisely the failure the quorum of section 7.4 exists to prevent.

Liveness is therefore decided by an explicit heartbeat, implemented in `vs_coord_membership`. Each coordinator announces itself to its peers once a second, and a peer that misses three announcements in a row is treated as gone. Three seconds to detect a partition instead of

Table 1: What can fail, how it is noticed, and what it costs.

What fails	How it is noticed	What happens	What is lost
Coordinator, crash	<code>nodedown</code> , at once	election; the new leader rebuilds from the stations	nothing
Coordinator, partition	heartbeat, 3 s	the minority suspends itself	new reservations, minority side
Station node, crash	<code>monitor_node</code> , at once	the coordinator drops that station's claims	sessions running there
Station uplink, partition	<code>monitor_node</code> , on the tick	the site keeps charging, starts nothing new	the operator's view
Charge point	socket closes, then 30 s grace	session closed with the last measured energy	nothing measured
Back office	nothing, it is a hidden node	the cluster is unaffected; it re-reads on restart	nothing durable
MySQL	connection errors	not tolerated, see section 8.8	writes, until it returns

sixty. `monitor_nodes` is subscribed to as well, but only to react faster when it does fire: it can never conclude anything the heartbeat would not have concluded a moment later.

Two details of the implementation are worth a sentence each. The heartbeat is sent with a plain asynchronous send to a registered name, which never blocks and never fails on an unreachable node, so no dead peer can slow down the tick that is supposed to notice it. And the state starts optimistic, with every peer assumed alive: starting pessimistic would mean that after a cluster-wide restart every node believes itself alone for three seconds, and three coordinators would elect themselves at the same moment.

Measured on 1 September, isolating the leader with `docker network disconnect`:

- QUORUM LOST (1 of 3) ... abdicating at **2.12 s**;
- a new leader serving with both claims at **2.29 s**;
- the distribution's own `nodedown` at **64.6 s**.

That last number is the argument. Without the heartbeat the system would have been incorrect for a full minute, and no test would have revealed it, because `docker kill`, the obvious way to simulate a failure, closes the socket and never exercises this path at all. Two kinds of failure need two experiments, which is why the demonstration shows both `kill` and `disconnect`.

8.3 A station dies, and a station goes silent

There are two distinct failures here, and conflating them is the most visible mistake available in this part of the system.

The node dies. `vs_coord_srv` takes an `erlang:monitor_node/2` on every station it has heard from, and on `nodedown` it erases that station's claims.

Doing nothing here would be the intuitive choice and it would be wrong. A claim is a promise that a vehicle is committed *somewhere*. If the somewhere no longer exists, the promise locks the driver out of the whole network until the lease expires: up to fifteen minutes of being unable to reserve anywhere, because of a failure they did not cause. Dropping the claims turns a station failure into a local one.

We state the cost. Charging sessions on that station are lost, because the process that was metering them is gone and no row was written. A *restart* does preserve the energy figure, because the charge point is the side that counts it and brings its running total back in the message it re-sends on reconnect. What comes back is the measurement. The session itself is over.

The uplink is cut. The more interesting failure, and the one with a plain meaning outside a classroom, is a station that is alive and unreachable. The car park is delivering power and the operator cannot see it.

Measured on 3 September, with `station2` isolated by `docker network disconnect`. The cars kept charging: energy climbed inside the isolated station from 0.248 to 0.403 to 0.558 kWh on one connector, and from 14.069 to 14.208 kWh on another. The coordinator logged `Node vs@station2 not responding`, dropped that station and its claims, and the lobby emptied, because the lobby is drawn from what the coordinator pushes. On reconnection the station announced itself again and neither session had been interrupted.

Two details sharpen the picture. The published ports survive, so port 9102 still answered 426 `Upgrade Required` throughout: a driver physically at the site could still open its page and use it. What is lost is the operator's view, which is the honest shape of this failure and is exactly why reservations expire on their own instead of waiting for somebody to cancel them.

But no new session can start. Authorising a car means resolving vehicle to owner, and that is a MySQL query across the severed link. `vs_cp_proto` logs `no account for vehicle N` and opens nothing, while charges already running are untouched, because they never touch the database. An isolated site finishes what it started and accepts nobody new. A real site controller with an unknown card and no uplink behaves the same way.

8.4 Rebuilding the claim table after a failover

A new leader inherits nothing. The claims were granted by its predecessor and lived in that process's memory, which died with it, so before granting anything it has to find out which vehicles are already held.

It does so by asking. The coordinator sends `who_do_you_hold` to every connected node that is not a coordinator, and each station walks its connectors and replies with those in `held` or `charging`, from memory. The leader switches from `rebuilding` to `serving` once the answers are in. Requests received in the meantime are refused with a retry indication, which the stations map onto a temporary refusal. The driver waits a second and then gets their connector.

Why asking is enough. This is the decision the whole failover story rests on. **The coordinator does not own the state it holds; it is an index over it.** The authoritative copy of "connector 3 is held for vehicle 88 until 15:42" lives on the station, which is also the only party that can act on it. The coordinator holds a derived view whose only purpose is to make the uniqueness check of section 7.2 a local lookup, and a derived view can be rebuilt from its sources.

That is why there is no replicated log, no consensus on a state machine, and no persistence at the coordinator. Replicating the claims would mean maintaining a second copy of something the stations already hold authoritatively: more machinery, more ways for two copies to disagree, and nothing gained. If the coordinator held information derivable from nowhere else, a replicated log would be unavoidable. It was designed so that it does not, and that is worth stating as a choice, because it was one.

Two paths converge. The explicit query is the fast path. There is a slower one that works even when the query reaches nobody: renewals arrive every ten seconds carrying the claim identifier and its original grant timestamp, and a renewal for an unknown claim is adopted rather than refused. The system would converge on its own within one renewal interval. The query makes recovery take a second instead of ten.

A hole we measured, and closed. The two paths were not enough on their own, and the defect is instructive because neither number involved was wrong by itself.

A coordinator that rejoins after a partition elects itself before its connections to the stations have been re-established. Its rebuild then asks nobody, receives nothing, and concludes in a microsecond that the network holds no claims. On 1 September a vehicle obtained a *second* reservation at another station 272 ms after such a leader began serving, and the invariant stayed broken for 13.65 s until the first renewal re-presented the older claim. The defence in place waited 2000 ms for stations to speak, while renewals arrive every 10000 ms: it covered a fifth of a cycle. The rebuild window is tuned to failure detection and the renewal interval is tuned to the cost of traffic at rest. The defect was that the first was being used to defend against the second.

The fix has two halves, one on each side of the system. Stations now re-announce and renew immediately on **nodeup** instead of waiting for their next tick, which puts their claims in flight within a second of the network coming back. And a rebuild that returns *no stations* while the table is *also empty* no longer starts serving. That combination does not describe an idle network: it is the signature of a leader that could not ask. The coordinator stays in **rebuilding** until a renewal tells it that the world exists.

The residual cost is declared. A rejoining leader refuses new reservations until it has heard from one station, which with the immediate renewal is a matter of milliseconds. The deadline timer is deliberately not cleared during that wait: it is the ceiling. If no renewal ever arrives, the backstop fires and the coordinator serves with a warning, because one that is stuck for ever is worse than one that is briefly wrong.

8.5 A charge point dies: grace, then honesty

The connector does not react to a lost socket immediately. A socket blip is not a broken charger, and a car charging perfectly well should not be stopped by our own reconnection logic. The **DOWN** arms a grace timer instead.

The window is one budget of ninety seconds, split in two: sixty seconds as the socket's idle timeout, which is two missed heartbeats at the thirty-second interval the charge-point protocol specifies, and the last thirty seconds as the connector's own grace. Splitting one budget in two places is a source of bugs and was one: an earlier version let both sides start a full window, so three missed heartbeats became six, and a charge point that had gone quiet stayed reservable for three minutes.

If the grace expires mid-session, the session is closed with the last energy the charge point reported. That figure is kept because the charge point was the only side counting, so its last report is the best available truth. The connector then becomes **out_of_service**. It does not go back to **free**: an outlet with no hardware attached cannot serve anyone, and offering it would send a driver to a socket that will do nothing when they arrive.

Recovery needs no operator. A charge point that boots and reports itself available lifts the connector back to **free**.

8.6 Supervision inside a node

Each supervision strategy encodes a claim about dependencies between processes, and the three supervisors in this system make three different claims. Choosing between them is a design decision of the same kind as the ones above.

- `vs_coord_sup` uses `rest_for_one`. The children are started in dependency order, so restarting one restarts everything downstream of it. When the process holding the claim table dies, everything derived from it must die too: a survivor holding references into a table that has been recreated is worse than a clean restart.
- `vs_station_sup` uses `one_for_one`, because its children are genuinely independent. The station manager crashing must not take down connectors that are delivering power to real cars.
- `vs_connector_sup` uses `simple_one_for_one` with `restart => transient`. Every child is the same kind of process, one per connector, and a connector that terminates normally stays terminated.

Two defects found while reviewing this code show that supervision only works if the rest of the code cooperates, and both were failures of supervision rather than of logic. The rebuild worker was started with `spawn_link` while the coordinator does not trap exits, so any exception inside it, including a malformed reply from any station on the network, would have killed the process holding every claim, and `rest_for_one` would then have taken the whole subtree down with it. It is now `spawn_monitor`, which delivers the same news without the lethality. Separately, the `rebuilding` state had exactly one exit, the success message: a worker that died before sending it left the coordinator refusing every reservation in the network, for ever, in silence. There is now a `DOWN` handler and a deadline, and both fall through to `serving` with a warning.

The general lesson is worth stating, because it applies well beyond this project. A supervision tree protects you from the failures you routed through it. A linked process and a state with no timeout are both ways of routing a failure around the tree.

8.7 Three delivery guarantees on one wire

The bridge from the coordinator to the back office carries three kinds of event, and they deliberately do not get the same guarantee, because the cost of a duplicate is different for each.

- **The session row: at-least-once over an idempotent write.** The back office prices a session with `UPDATE sessions SET cost_cents = ? WHERE id = ? AND cost_cents IS NULL`. A repeat updates zero rows and is not an error. At-least-once over an idempotent write is the standard way of obtaining effectively-once, and billing is also driven by a periodic sweep, so a lost message delays the invoice without changing what it says.
- **The no-show strike: at-most-once.** A counter cannot recognise a duplicate, and a doubled strike would suspend an innocent driver. One send, no retry. A lost strike is a smaller wrong than an invented one.
- **The notification: at-most-once.** A duplicate is a row somebody reads twice; a loss is a row they never see. Neither is good, and we chose the quieter failure.

Choosing the guarantee per message rather than per channel is the point. A single reliable-messaging layer would have forced one answer onto three questions that have different answers.

8.8 What tolerates nothing, and is declared

Three things in this system have no recovery path, and we prefer to name them than to let them be discovered.

MySQL is a single point of failure for durable data. Replicating it was out of reach for this project. The honest position is not that every single point of failure has been eliminated, which is not true of any system this size, but that we chose which ones to keep and know what happens when they fail. Note what does *not* depend on the database: charging continues, power sharing continues, and the coordinator keeps granting claims, because none of them reads MySQL on the interactive path.

Sessions on a dead station are lost, as section 8.3 describes. The process that was metering them is gone.

The rebuild can still miss a station that is unreachable at exactly the wrong moment. Its claims are absent from the new leader's table, so in principle one of its vehicles could obtain a second claim elsewhere before the conflict is noticed. In practice a car plugged in at that station is not in another city, and the conflict is resolved when the station returns and re-presents the older claim, which wins. The window is narrow and it is real, and it is declared here instead of being left for somebody to find.

9 API and Protocol Specification

[to be written — 4 pages. One message table per channel: name, direction, payload, effect, errors. A table beats a listing here, it fits in less space and can be consulted. 9.1 HTTP endpoints of the back office, with the JWT note (contracts/jwt.md); 9.2 driver WebSocket (contracts/ws-driver.md); 9.3 charge-point WebSocket (contracts/ws-chargepoint.md), declaring which OCPP 1.6-J messages are implemented and which are not; 9.4 claim protocol (contracts/claim.md); 9.5 JInterface bridge (contracts/erlang-java.md). One sequence figure for the reservation with claim, including the refusal path.]

10 Deployment

[to be written — 2 pages. Seven nodes, the containers, the networks, how the system is started. Taken from deploy/docker-compose.yml, not from memory. State precisely that the deployment runs on one host with seven Erlang and JVM nodes, and that a genuine network partition is produced with `docker network disconnect` rather than with a second machine. Include the per-site networks, which exist to make a site partition demonstrable. Sources: deploy/, DEMO.md.]

11 Testing and Demonstration

[to be written — 2 pages. What is tested automatically (EUnit, scripts/eunit_check.sh, with the expected test count) and what is shown live. The scenarios are in DEMO.md: contention on one connector, cross-node claim, no-show, power sharing in real time, coordinator failover, station crash, station partition. For each one: what appears on screen and which problem of section 5 it demonstrates. That mapping is what turns a demo into evidence.]

12 Future Works

[to be written — 1 page. Site power shared across stations on the same grid connection; partitioning the vehicle space across coordinators; operator dashboard with the ability to take connectors out of service; native push notifications, which the overstay notice needs. Short and honest: these are things not done, not promises. Sources: SCOPE §10, DESIGN-NOTES §7.]